



4



8



960 sp

عملي

كلية الهندسة المعلوماتية

السنة الثالثة

Texture

عمل طلابي

محتوى مجاني غير مخصص للبيع التجاري

10/11/2024

بيانات حاسوبية

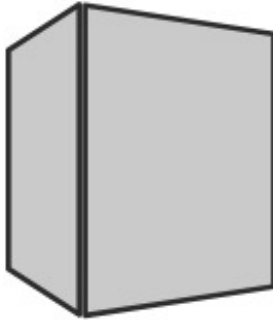


RB Informatics

Texture

ما هو الـ texture؟

هو وضع صورة على شكل للحصول على منظر واقعي أكثر، مثال:



Before using Texture



After using Texture

هناك طرق مختلفة لتطبيق الـ Texture في OpenGL:

بعض الطرق تعتمد على تقنية Bitmapping.

الطرق الأخرى تعتمد على استخدام صور جاهزة، وهذه هي الطريقة التي سنركز عليها في هذه المحاضرة.

تهيئة OpenGL لاستقبال الـ Texture:

قبل البدء بتطبيق الـ Texture على الأشكال، يجب إعداد بيئة OpenGL لاستقبال الصور.

ملاحظة: يمكنكم الحصول على المكتبات اللازمة عبر مسح الـ QR code التالي:

■ إعداد ملفات ال Texture:

🔥 ضع ملف texture.h في المسار التالي إذا كنت تستخدم vs code:

"C:\Program Files (x86)\Microsoft Visual Studio 11.0\VC\include"

🔥 أو في المسار التالي إذا كنت تستخدم clion:

"C:\Program Files\JetBrains\CLion 2022.1.1\bin\mingw\x86_64-w64-mingw32\include"

🔥 أو يمكنك تضمينه في المشروع ووضعه داخل مجلد ال headers كما شرحنا في الملحق.

ثم ضع ملف texture.cpp في المشروع الذي ستستخدم فيه ال Texture في المجلد source كما شرحنا في الملحق.

■ ملاحظات حول الصور:

■ يجب أن تكون الصورة بامتداد .bmp.

■ يمكننا تحويل أي صورة إلى BMP (امتداد .bmp)، باستخدام الرسام Microsoft Paint (على نظام Windows)

■ افتح برنامج Paint الموجود بشكل افتراضي على جميع أنظمة Windows.

■ انقر على File (ملف) ثم Open (فتح) وحدد الصورة التي تريد تحويلها.
لحفظ الصورة كـ BMP:

■ بعد فتح الصورة، انقر على File (ملف).

■ اختر Save As (حفظ باسم).

■ من قائمة Save as type (نوع الملف)، اختر BMP Picture (.bmp).

■ اختر المجلد الذي تريد حفظ الصورة فيه، ثم اضغط على Save (حفظ).

■ يُفضل أن تكون أبعاد الصورة من قوى العدد 2 (مثل 128×256 أو 64×64).

ملاحظة: في حال وجود مشاكل بالصورة عند تنفيذ الكود، غالباً ما تكون المشكلة من خطوة تحويل الامتداد، ويجب تعديل الأبعاد كما في الخطوة الأخيرة والمحاولة مجدداً.

خطوات تطبيق ال Texture في OpenGL:

الخطوة 1: تفعيل ال Texture

يجب تفعيل خاصية ال Texture في OpenGL باستخدام التابع:

`void glEnable(GLenum parameter); // Enable using textures in the scene`

حيث ال parameters المسموحة هي: `GL_TEXTURE_1D`, `GL_TEXTURE_2D`, `GL_TEXTURE_3D` وسنركز في محاضرتنا على `GL_TEXTURE_2D` فقط، فنكتب التابع بالشكل:

`glEnable(GL_TEXTURE_2D); // Enable 2D Textures`

ويكفي استدعاء التابع لمرة واحدة وبالتالي يمكننا وضع هذه التعليمة في ال `InitGL`.

الخطوة 2: Load Texture

لتنفيذ هذه الخطوة، يتم استخدام التابع:

```
int LoadTexture(char* filename, int alpha = 255); // Load the texture image to the scene
```

الـ **parameter الأول**: filename وهو مسار الملف الذي يحتوي على الصورة.

الـ **parameter الثاني**: alpha لتحديد الشفافية (القيمة الافتراضية هي 255 وكلما نقصت كلما زادت شفافية الصورة) يُعيد هذا التابع قيمة int التي تمثل عنوان الصورة في الذاكرة، وبالتالي يجب تخزين هذه القيمة في متغير لأننا سنحتاجها في الخطوة 3.

ملاحظة: يفضل وضع الصورة المراد استخدامها في مجلد الـ source الخاص بملف المشروع وذلك لكي يعمل الكود على جميع الأجهزة حتى لو تم نقل المشروع على جهاز آخر، ويمكن القيام بهذه العملية بنسخ الصورة المطلوبة ثم الضغط باليمين على الملف source ولصقها (والانتباه أن لاحقتها يجب أن تكون bmp)، وبعد اتباع هذه الخطوات يكفي كتابة اسم الصورة مع لاحقتها في التابع دون الحاجة لكتابة مسارها الكامل بدءاً من الـ C. في حال لم يتم تطبيق الصورة على الشكل بعد تطبيق هذه الخطوات، فيجب التأكد من أن الصور قد تم نسخها إلى ملف المشروع بشكل صحيح وذلك عن طريق فتح مجلد المشروع والتأكد من وجود الصور فيه.

مثال:

```
// Before InitGL function
```

```
int image;
```

```
// Inside InitGL function
```

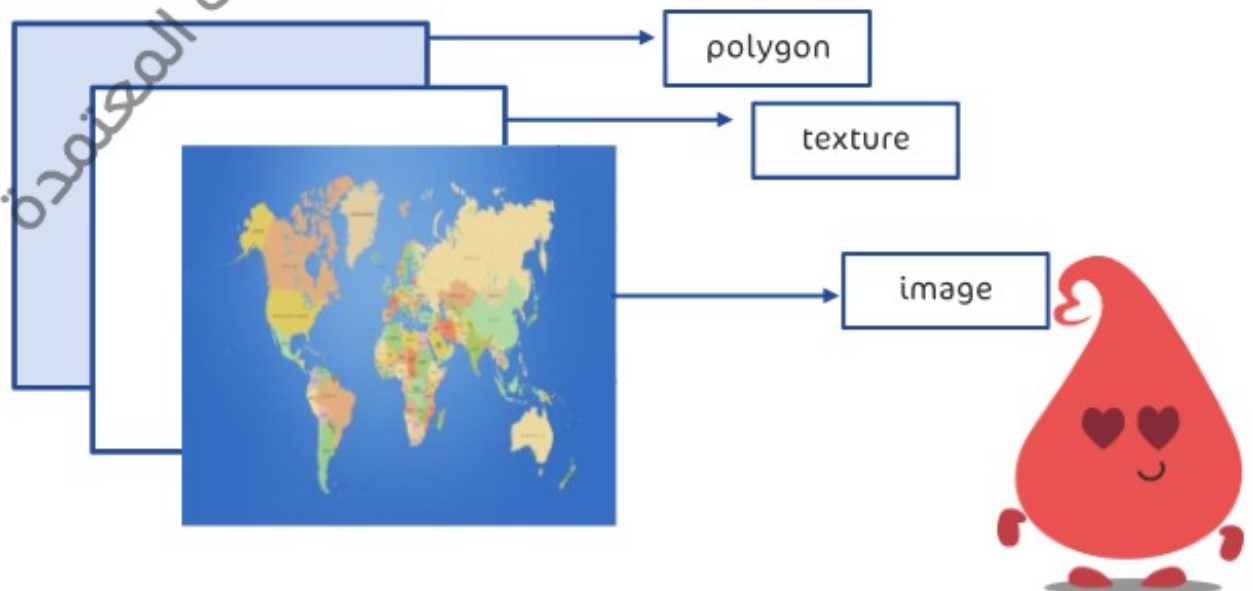
```
image = LoadTexture("texture_image.bmp"); // Load the texture image
```

يكفي استدعاء التابع لمرة واحدة وبالتالي نضع هذه التعليمة في الـ InitGL.

الخطوة 3: Bind Texture

ما هو الـ Texture؟؟

هو مربع طول ضلعه يساوي الواحد وله وجهان، يتم لصق الصورة المراد استخدامها على الوجه الأول، ويتم لصق الوجه الثاني من الـ texture على الـ polygon، فيبدو وكأن الصورة الأساسية وضعت على الـ polygon.



للقيام بالمرحلة الأولى (لصق الصورة على الوجه الأول) نستخدم التابع:

```
void glBindTexture(GLenum target, GLuint texture);
```

الـ target: سنستخدم في محاضراتنا على GL_TEXTURE_2D فقط.

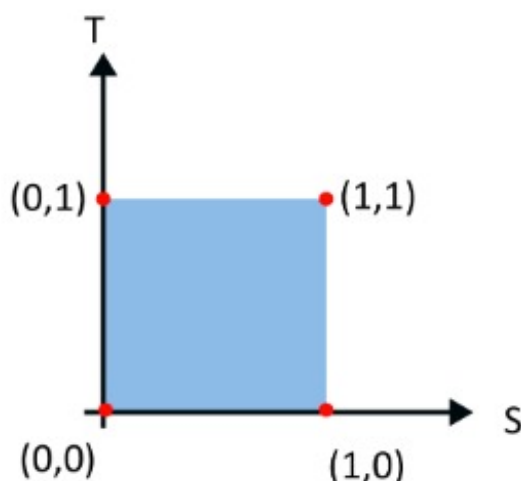
الـ texture: هو المتغير image الذي خزناه في الخطوة 2 والذي هو خرج التابع LoadTexture.

يتم استخدامه في تابع DrawGLScene بهذا الشكل:

```
glBindTexture(GL_TEXTURE_2D, image); // Bind the texture for use
```

في هذه الخطوة، نقوم بلصق الصورة على الـ Texture.

ملاحظة: الـ texture موجود في نظام إحداثيات 2D أي أنه في عالم غير عالم OpenGL (3D)، حيث أن المحاور الإحداثية لـ OpenGL هي (x, y, z) بينما سنرمز لمحور الفواصل الخاص بالـ texture بـ s و s و t لترتيبها كما في الشكل المجاور:

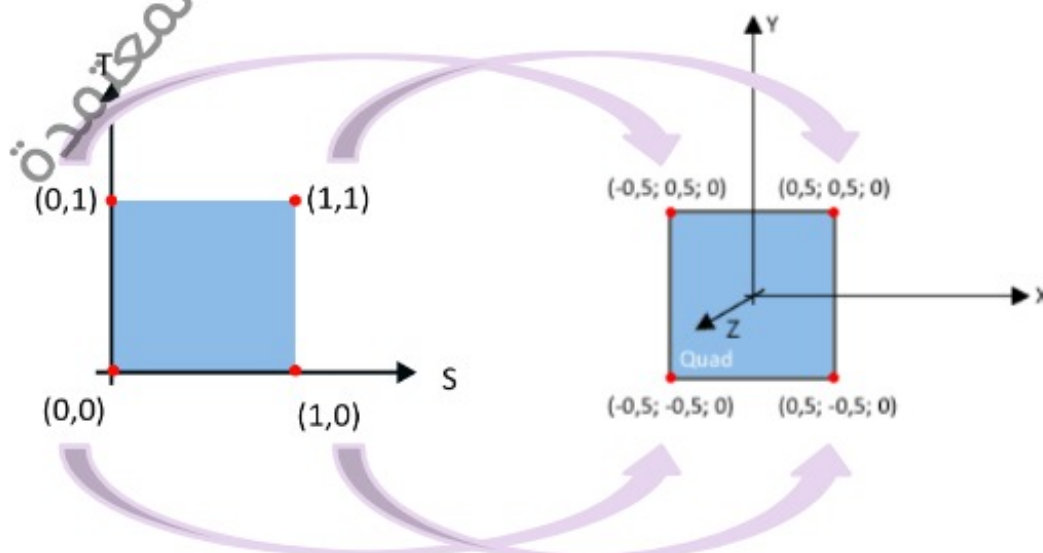


الخطوة 4: Mapping Texture Coordinates

للقيام بالمرحلة الثانية (لصق الوجه الثاني للـ texture على الـ polygon)، نقوم بربط كل نقطة من الـ texture بنقطة مقابلة لها في الـ polygon. يُطلق على هذه العملية اسم Mapping، حيث يتم إقران كل نقطة من الـ texture بموقع معين في الـ polygon، ويقوم بهذه المهمة التابع:

```
void glTexCoord2d(GLdouble S, GLdouble T);
```

حيث (S, T) هي إحداثيات لنقطة ما على الـ texture، وبالتالي يجب أن نكتب التابع مرة من أجل كل نقطة نريد تحديدها من نقاط الـ texture (ويكتب التابع في DrawGLScene)، ولتحديد النقطة المقابلة لها على الـ polygon نكتب التتابع على شكل ثنائيات، أولاً نحدد النقطة على الـ texture ثم نقوم برسم النقطة المقابلة لها على الـ polygon، وهكذا لجميع نقاط الشكل.



مثال على تطبيق Texture على شكل رباعي (Quad):

```

int img; // InitGL نعرف المتغير قبل التابع
int InitGL(GLvoid) {
    // نضيف هذه الأسطر على التابع دون حذف الكود المكتوب به سابقاً
    glEnable(GL_TEXTURE_2D); // تفعيل الـ texture
    img = LoadTexture("texture_image.bmp", 255); // تخزين عنوان الصورة في img
    return TRUE;
}

void DrawGLScene(GLvoid) {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();

    glTranslated(0, 0, -10);
    glBindTexture(GL_TEXTURE_2D, img);

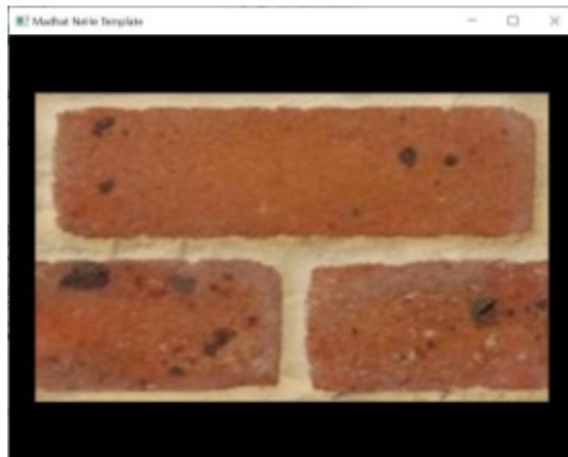
    glBegin(GL_QUADS);
    // تحديد رؤوس المربع والنقاط المقابلة لكل رأس على الـ texture
    glTexCoord2f(0.0f, 0.0f);
    glVertex3f(-5.0f, -3.0f, 0.0f); // Bottom-left corner

    glTexCoord2f(0.0f, 1.0f);
    glVertex3f(-5.0f, 3.0f, 0.0f); // Top-left corner

    glTexCoord2f(1.0f, 1.0f);
    glVertex3f(5.0f, 3.0f, 0.0f); // Top-right corner

    glTexCoord2f(1.0f, 0.0f);
    glVertex3f(5.0f, -3.0f, 0.0f); // Bottom-right corner
    glEnd();
    glFlush();
    SwapBuffers(hdc);
}

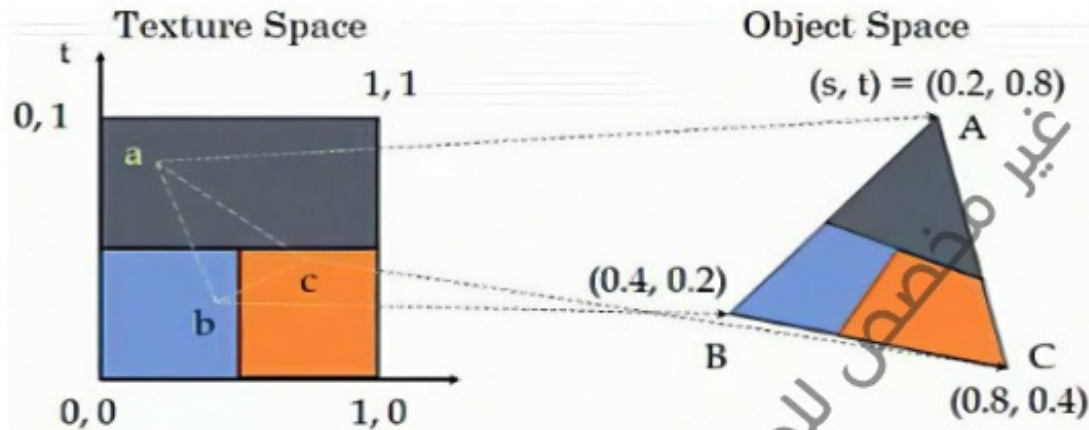
```



ماذا لو كان الشكل غير رباعي؟

إحداثيات الـ texture التي نمررها للتابع `glTexCoord2f` لا يجب أن تكون دائماً (0 أو 1). يمكن أن تكون أي قيمة بينهما حسب توزيع الصورة على الشكل.

مثال:



```
glBegin(GL_TRIANGLES); // Start drawing a triangle
glTexCoord2f(0.0f, 0.0f); glVertex3f(-2.0f, -2.0f, 0.0f);
// Bottom-left
glTexCoord2f(0.5f, 1.0f); glVertex3f(0.0f, 2.0f, 0.0f);
// Top-center
glTexCoord2f(1.0f, 0.0f); glVertex3f(2.0f, -2.0f, 0.0f);
// Bottom-right
glEnd();
```



Repeating Texture

Repeating Texture هو عملية يتم فيها تكرار الصورة (أو النمط) على سطح معين بدلاً من مد الصورة لتغطيته بالكامل.

لماذا نستخدم التكرار؟

إذا كانت لدينا صورة صغيرة ونريد تطبيقها عدة مرات على سطح كبير، بدلاً من تمديد الصورة لتغطي السطح بالكامل (مما قد يجعلها غير واضحة)، يمكننا تكرار الصورة بحيث يتم تغطية السطح بأجزاء مكررة منها، ونستفيد منه لرسم الأرضيات أو الجدران التي تحوي على نمط مكرر.

كيفية تفعيل الـ (Repeating Texture):

لتكرار الصورة على السطح في OpenGL، نستخدم تابعًا خاصًا لتحديد آلية التعامل مع الـ Texture في حال تجاوز حدود الصورة الأصلية. (أي في حال مررنا في التابع `glTexCoord2f` قيم أكبر من الواحد)، هذا التابع هو `glTexParameteri`.

الخطوات:

لتكرار الصورة على المحور X:

نستخدم التابع التالي لمرة واحدة في `InitGL`:

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
```

`GL_TEXTURE_WRAP_S` يمثل المحور X (الاتجاه الأفقي). `GL_REPEAT` يُخبر OpenGL بتكرار الصورة عندما تتجاوز الحدود في هذا الاتجاه.

لتكرار الصورة على المحور Y:

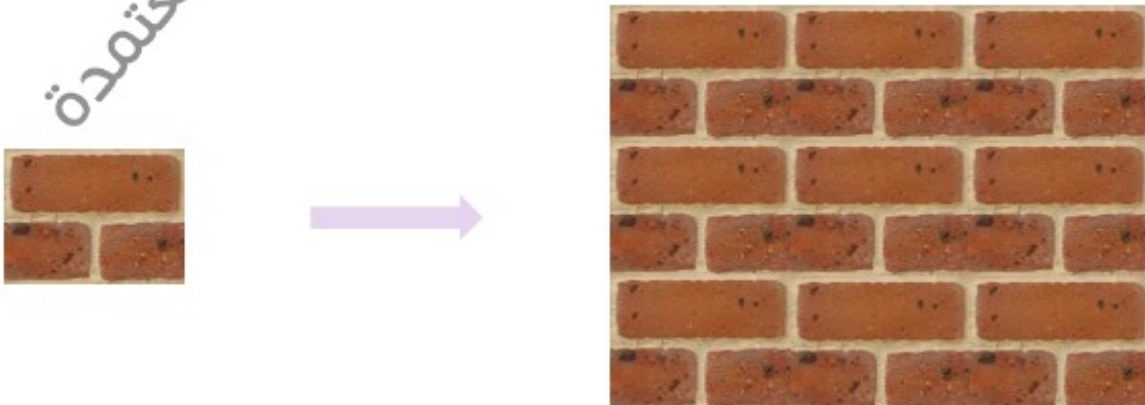
نستخدم التابع التالي لمرة واحدة في `InitGL`:

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
```

`GL_TEXTURE_WRAP_T` يمثل المحور Y (الاتجاه العمودي). `GL_REPEAT` يُخبر OpenGL بتكرار الصورة عندما تتجاوز الحدود في هذا الاتجاه.

مثال لتطبيق الـ Repeating texture:

لنفترض أننا نريد رسم حائط يتكون من مربعات مكررة، حيث كل مربع يمثل جزءًا مكررًا من الصورة:

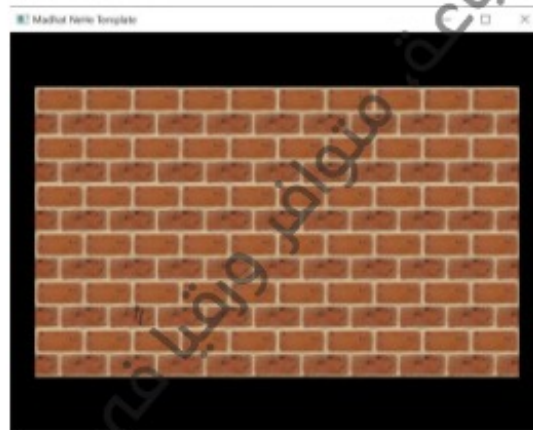


// InitGL تفعيل تكرار الصورة على المحورين x و y في التابع

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
```

// DrawGLScene رسم مربع كبير مع تكرار الصورة عليه في

```
glBegin(GL_QUADS);
glTexCoord2f(0.0f, 0.0f); glVertex3f(-5.0f, -3.0f, 0.0f);
// bottom-left corner
glTexCoord2f(0.0f, 6.0f); glVertex3f(-5.0f, 3.0f, 0.0f);
// top-left corner
glTexCoord2f(10.0f, 6.0f); glVertex3f(5.0f, 3.0f, 0.0f);
// top-right corner
glTexCoord2f(10.0f, 0.0f); glVertex3f(5.0f, -3.0f, 0.0f);
// bottom-right corner
glEnd();
```



شرح المثال:

glTexCoord2f(0.0f, 0.0f)

هذه النقطة هي الزاوية السفلية اليسرى للـ texture. نضعها عند الزاوية السفلية اليسرى للشكل.

glTexCoord2f(0.0f, 6.0f)

هذه النقطة تمثل تكرار الصورة 6 مرات في الاتجاه العمودي (المحور Y).

glTexCoord2f(10.0f, 6.0f)

هذه النقطة تمثل تكرار الصورة 10 مرات في الاتجاه الأفقي (المحور X) و 6 مرات في الاتجاه العمودي (المحور Y).

glTexCoord2f(10.0f, 0.0f)

تكرار الصورة 10 مرات في الاتجاه الأفقي فقط.



The End